



RESEARCH BOOK

SUPPLY CHAIN ANALYTICS DASHBOARD

– Using Python and Streamlit –



SUBMITTED BY:

Karan Kumar Soni

gk3918429@gmail.com

Department of Computer Science & Engineering
Dr. A. P. J. Abdul Kalam Technical University, Lucknow
Academic Year :- 2023–2027

UNDER THE GUIDANCE OF:

UNIFIED MENTOR

"Data Driven Insights • Smarter Decisions • Stronger Supply Chains"

Abstract :-

Supply chain management plays a crucial role in ensuring the efficient movement of goods, timely deliveries, and customer satisfaction. With the rapid growth of e-commerce and global trade, organizations generate a massive amount of logistics data that requires effective analysis for better decision-making. This project presents a **Supply Chain Analytics Dashboard** developed using **Python, Streamlit, Pandas, and Plotly** to analyze and visualize supply chain performance through an interactive web-based interface.

The dashboard enables users to monitor key performance indicators (KPIs), including the total number of orders, average delivery gap, delayed orders, and late delivery percentage. It also provides interactive filters based on shipping mode and order region, allowing users to explore data according to their specific requirements. Various visualizations, such as bar charts, pie charts, and histograms, help identify shipping trends, delivery status, and regional performance, making the analysis more intuitive and effective.

The project uses the **Data Co Supply Chain Dataset**, where data preprocessing techniques are applied to calculate delivery gaps and generate meaningful business insights. In addition, the dashboard allows users to preview filtered data and download it in CSV format for further analysis.

The developed system demonstrates how data analytics and visualization can improve supply chain transparency, identify operational bottlenecks, and support data-driven decision-making. In the future, this dashboard can be enhanced by integrating machine learning models for delivery delay prediction, demand forecasting, inventory optimization, and real-time data integration. Overall, this project provides a practical and scalable solution for modern supply chain performance analysis.

Keywords :-

- Supply Chain
- Streamlit
- Python
- Pandas
- Plotly
- Dashboard
- Data Analytics

INTRODUCTION

1.1 Supply Chain Analytics :-

Supply Chain Analytics is the process of collecting, analyzing, and visualizing supply chain data to improve operational efficiency, reduce costs, and support better business decisions. In today's competitive business environment, organizations generate a large amount of logistics and transportation data every day. Managing this data manually is time-consuming and often leads to delays in identifying operational issues. Therefore, data analytics has become an essential tool for optimizing supply chain performance and improving customer satisfaction.

This project presents a **Supply Chain Analytics Dashboard** developed using **Python, Streamlit, Pandas, and Plotly**. The dashboard provides an interactive platform for analyzing logistics data through dynamic charts, key performance indicators (KPIs), and filtering options. Users can monitor total orders, average delivery gaps, delayed deliveries, and shipping performance across different regions and shipping modes.

The dashboard is built using the **DataCo Supply Chain Dataset**, which contains order, shipping, customer, and delivery information. The data is processed using the Pandas library, while Plotly is used to generate interactive visualizations. Streamlit is used to create a responsive web application that allows users to interact with the dashboard in real time.

The main objective of this project is to simplify supply chain data analysis by presenting meaningful insights through an easy-to-use interface. The dashboard helps managers and decision-makers identify operational bottlenecks, monitor delivery performance, and make data-driven decisions to improve the efficiency of the supply chain. Furthermore, the project demonstrates how modern data analytics tools can transform raw logistics data into valuable business intelligence.

1.2 Need of Dashboard :-

In today's business environment, supply chain operations generate a huge amount of data related to orders, shipping, inventory, transportation, and customer deliveries. Analyzing this data using traditional spreadsheets or manual reports is time-consuming and makes it difficult to identify important trends and operational issues. Therefore, there is a growing need for an interactive dashboard that can present complex data in a simple and meaningful way.

A Supply Chain Analytics Dashboard helps managers and decision-makers monitor key performance indicators (KPIs) such as total orders, delivery delays, shipping performance, and regional distribution in real time. By using charts, graphs, and interactive filters, users can quickly identify delays, compare shipping modes, and evaluate logistics performance across different regions.

The dashboard also reduces the time required for data analysis by automatically processing large datasets and displaying actionable insights. This enables organizations to make faster, data-driven decisions that improve operational efficiency, reduce delivery delays, enhance customer satisfaction, and optimize supply chain performance.

In this project, the dashboard is developed using **Python, Pandas, Plotly, and Streamlit**, providing an easy-to-use web interface for analyzing supply chain data. It offers interactive visualizations, filtering options, and downloadable reports, making it a practical solution for modern supply chain management.

1.3 Existing Problems :-

Supply chain management involves multiple processes such as procurement, inventory management, transportation, warehousing, and product delivery. Many organizations still rely on traditional methods such as spreadsheets, manual reports, and disconnected systems to monitor these operations. As the volume of logistics data continues to increase, these methods become inefficient and make it difficult to identify important business insights.

One of the major problems in existing supply chain systems is the **lack of real-time visibility** into logistics operations. Managers often find it difficult to track delivery performance, monitor shipping delays, and compare operational efficiency across different regions. This results in delayed decision-making and reduced productivity.

Another significant challenge is the **manual analysis of large datasets**. Analyzing thousands of order records using spreadsheets requires considerable time and effort and increases the possibility of human errors. It also becomes difficult to identify patterns, trends, and exceptions without proper visualization tools.

Traditional reporting systems also provide **limited interactive capabilities**. Users cannot easily filter data based on shipping mode, region, or delivery status, making detailed analysis more complicated. As a result, organizations may fail to detect operational bottlenecks and opportunities for improvement.

To overcome these challenges, there is a need for an interactive and user-friendly analytics dashboard that provides real-time insights through visualizations, key performance indicators (KPIs), and filtering options. The proposed **Supply Chain Analytics Dashboard** addresses these problems by transforming raw logistics data into meaningful visual information, enabling faster decision-making, improved delivery performance, and more efficient supply chain management.

Objectives :-

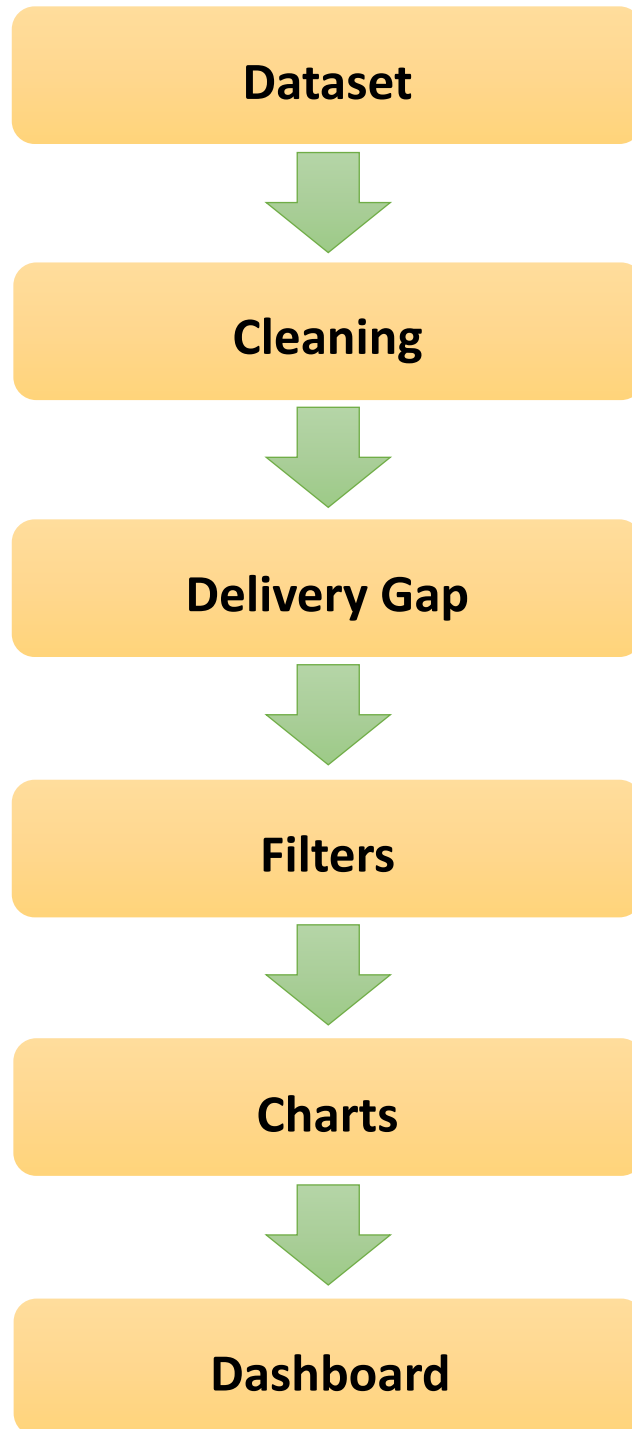
- Analyze Orders
- Delivery Delay
- Shipping Mode
- Region Analysis
- Download CSV

TECHNOLOGY STACK

<u>Technology</u>	<u>Purpose</u>
Python	Programming
Pandas	Data Analysis
Plotly	Charts
Streamlit	Dashboard
GitHub	Version Control

METHODOLOGY

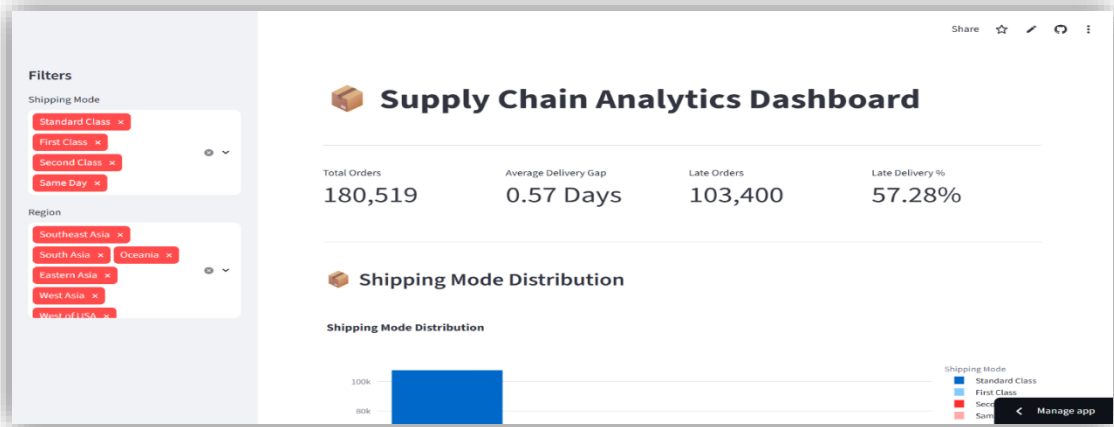
- Flow Diagram :-



Dashboard Implementation



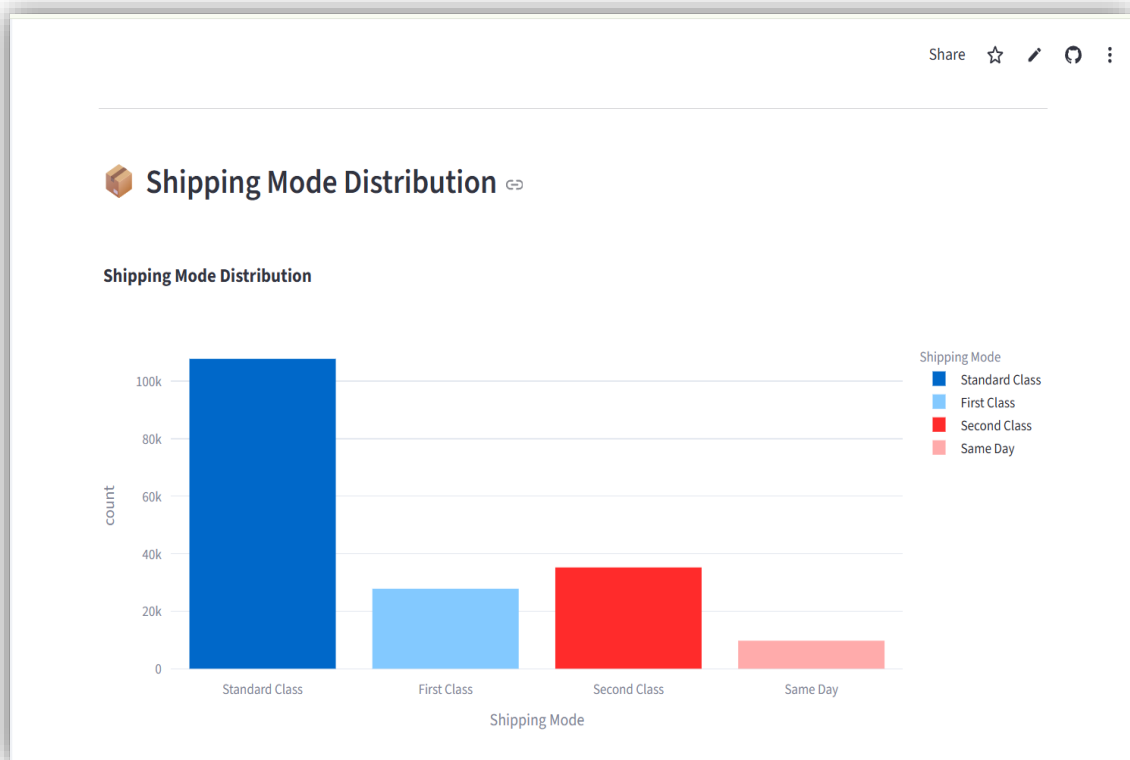
Dashboard Home :-



The Dashboard Home successfully displays the key performance indicators (KPIs) of the supply chain, including Total Orders, Average Delivery Gap, Late Orders, and Late Delivery Percentage. It also provides interactive filters for Shipping Mode and Region, enabling users to analyze data efficiently and make informed decisions.



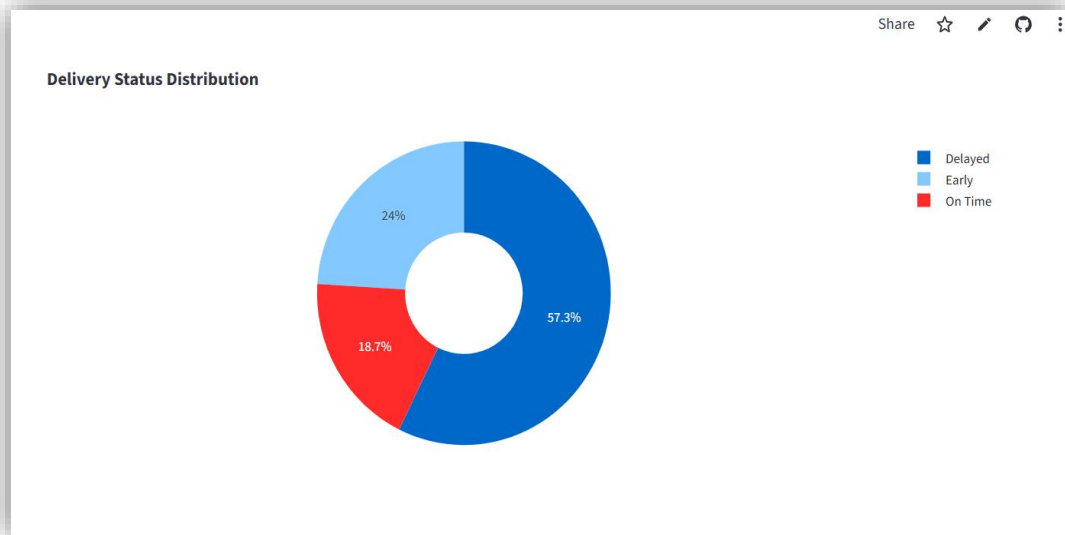
Shipping Mode Distribution :-



The Shipping Mode Distribution chart shows that **Standard Class** is the most frequently used shipping method, while **Same Day** shipping has the lowest number of orders. This analysis helps compare different shipping modes and supports better logistics planning and operational decision-making.



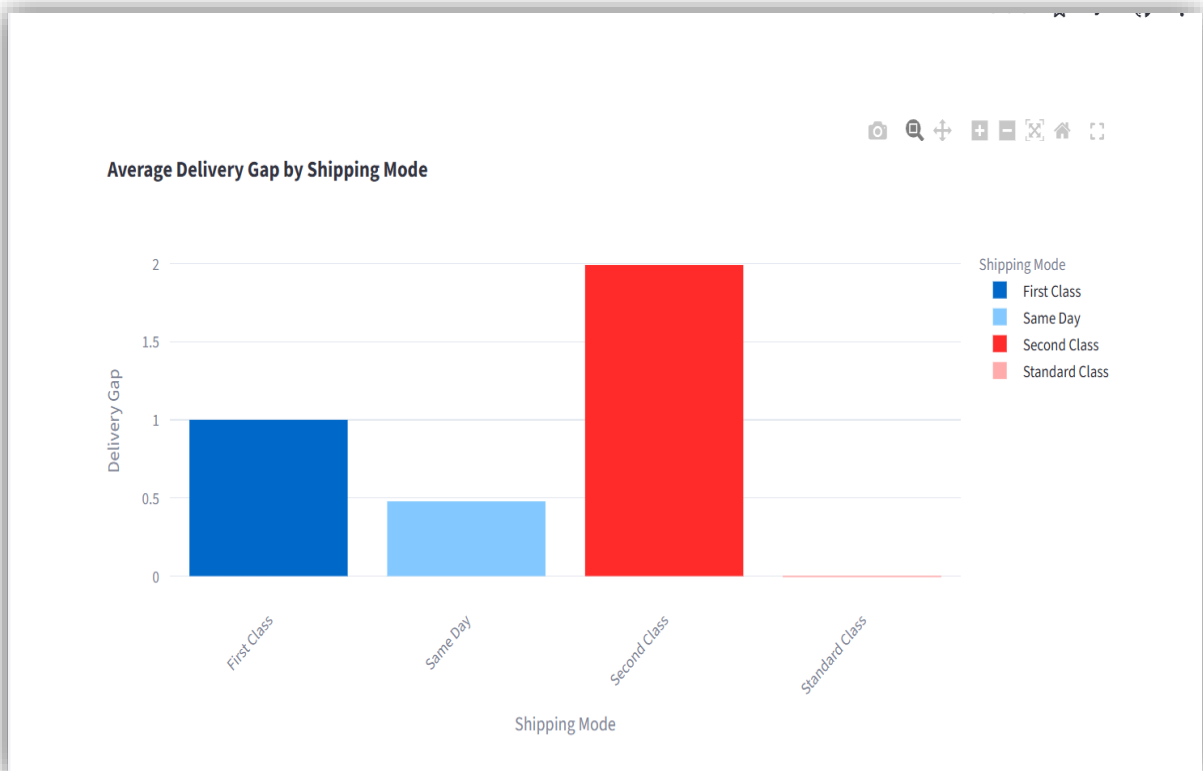
Delivery Status Distribution :-



The Delivery Status Distribution chart classifies orders into **Early**, **On Time**, and **Delayed** categories. It helps evaluate the overall delivery performance of the supply chain and identify delays that require operational improvements. This analysis supports better decision-making and enhances customer satisfaction.

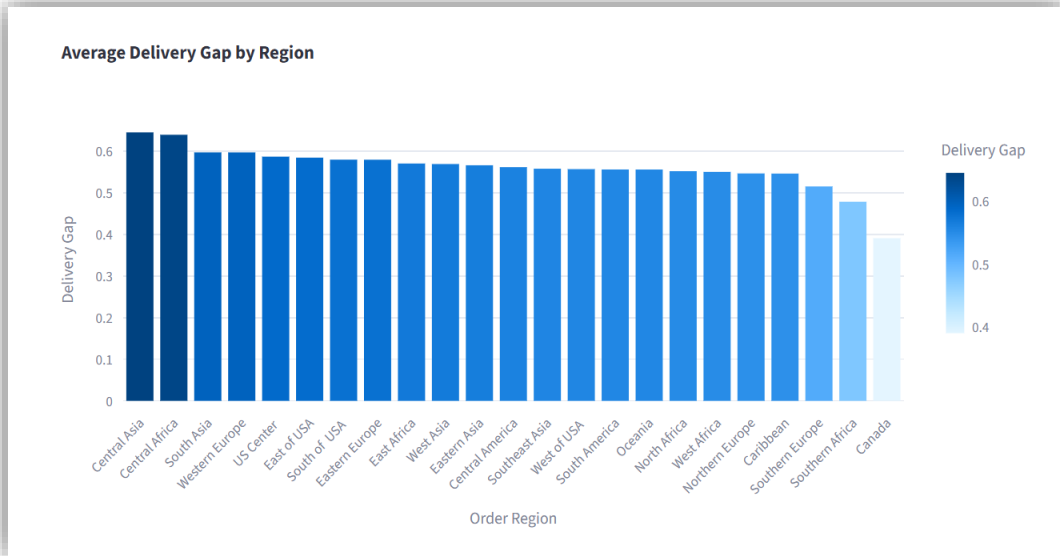


Average Delivery Gap by Shipping Mode :-



The Average Delivery Gap by Shipping Mode chart compares the delivery performance of different shipping methods. It helps identify which shipping mode delivers orders more efficiently by showing the average difference between scheduled and actual delivery time. Shipping modes with lower delivery gaps indicate better performance, while higher gaps highlight areas that require improvement.

❖ Average Delivery Gap by Region :-



The Average Delivery Gap by Region chart compares delivery performance across different regions. It helps identify regions with higher and lower delivery delays, allowing organizations to improve logistics planning, optimize resource allocation, and enhance overall supply chain efficiency.

Note: Report me heading "**Average Delivery Gap by Region**" likhna. "Reason" nahi, "**Region**" sahi word hai.

❖ Dataset Preview :-

Dataset Preview

	Type	Days for shipping (real)	Days for shipment (scheduled)	Benefit per order	Sales per customer	Delivery Status	Late_delivery_risk	Category Id	Category Name	Customer City	Customer Country	Customer Email	Customer
0	DEBIT	3	4	91.25	314.64	Early	0	73	Sporting Goods	Caguas	Puerto Rico	XXXXXXXXXX	Cally
1	TRANSFER	5	4	-249.09	311.36	Delayed	1	73	Sporting Goods	Caguas	Puerto Rico	XXXXXXXXXX	Irene
2	CASH	4	4	-247.78	309.72	On Time	0	73	Sporting Goods	San Jose	EE. UU.	XXXXXXXXXX	Gillian
3	DEBIT	3	4	22.86	304.81	Early	0	73	Sporting Goods	Los Angeles	EE. UU.	XXXXXXXXXX	Tana
4	PAYMENT	2	4	134.21	298.25	Early	0	73	Sporting Goods	Caguas	Puerto Rico	XXXXXXXXXX	Orli
5	TRANSFER	6	4	18.58	294.98	Delayed	0	73	Sporting Goods	Tonawanda	EE. UU.	XXXXXXXXXX	Kimberly
6	DEBIT	2	1	95.18	288.42	Delayed	1	73	Sporting Goods	Caguas	Puerto Rico	XXXXXXXXXX	Constance
7	TRANSFER	2	1	68.43	285.14	Delayed	1	73	Sporting Goods	Miami	EE. UU.	XXXXXXXXXX	Erica
8	CASH	3	2	133.72	278.59	Delayed	1	73	Sporting Goods	Caguas	Puerto Rico	XXXXXXXXXX	Nichole
9	CASH	2	1	132.15	275.31	Delayed	1	73	Sporting Goods	San Ramon	EE. UU.	XXXXXXXXXX	Oprah

Download CSV

The Dataset Preview section displays the filtered records in a table format, allowing users to verify and analyze the data easily. It also provides a **Download CSV** feature, enabling users to export the filtered dataset for further analysis and reporting.



VS Code Imports :-

```
app.py x requirements.txt
app.py > ...
1 import streamlit as st
2 import pandas as pd
3 import plotly.express as px
4 import plotly.graph_objects as go
5
6 # -----
7 # Page Configuration
8 # -----
9 st.set_page_config(
10     page_title="Supply Chain Dashboard",
11     page_icon="📦",
12     layout="wide"
13 )
14
15 st.title("📦 Supply Chain Analytics Dashboard")
16 st.markdown("----")
17
18 # -----
19 # Load Dataset
20 # -----
21 @st.cache_data
22 def load_data():
23     df = pd.read_csv("data/DataCoSupplyChainDataset.csv", encoding="latin1")
24     return df
25
26 df = load_data()
27
28 # -----
29 # Data Preparation
30 # -----
31 df["Delivery Gap"] = (
32     df["Days for shipping (real)"] -
33     df["Days for shipment (scheduled)"]
34 )
35
36 # -----
37 # Sidebar
38 # -----
39 st.sidebar.header("Filters")
40
41 shipping = st.sidebar.multiselect(
42     "Shipping Mode",
43     df["Shipping Mode"].unique(),
44     default=df["Shipping Mode"].unique()
45 )
46
47 region = st.sidebar.multiselect(
48     "Region",
49     df["Order Region"].unique(),
50     default=df["Order Region"].unique()
51 )
52
53 filtered_df = df[
54     (df["Shipping Mode"].isin(shipping)) &
55     (df["Order Region"].isin(region))
56 ]
57
58 # -----
59 # KPI Cards
60 # -----
61 total_orders = len(filtered_df)
62
63 avg_gap = filtered_df["Delivery Gap"].mean()
64
65 late_orders = len(filtered_df[filtered_df["Delivery Gap"] > 0])
66
67 late_ratio = (late_orders / total_orders) * 100
68
```



Sidebar Filters :-

```
app.py x requirements.txt
app.py > ...
35
36 # -----
37 # Sidebar
38 # -----
39 st.sidebar.header("Filters")
40
41 shipping = st.sidebar.multiselect(
42     "Shipping Mode",
43     df["Shipping Mode"].unique(),
44     default=df["Shipping Mode"].unique()
45 )
46
47 region = st.sidebar.multiselect(
48     "Region",
49     df["Order Region"].unique(),
50     default=df["Order Region"].unique()
51 )
52
53 filtered_df = df[
54     (df["Shipping Mode"].isin(shipping)) &
55     (df["Order Region"].isin(region))
56 ]
57
58 # -----
59 # KPI Cards
60 # -----
61 total_orders = len(filtered_df)
62
63 avg_gap = filtered_df["Delivery Gap"].mean()
64
65 late_orders = len(filtered_df[filtered_df["Delivery Gap"] > 0])
66
67 late_ratio = (late_orders / total_orders) * 100
68
```



Charts Code :-

```
app.py > ...
114
115 fig = px.bar(
116     gap,
117     x="Shipping Mode",
118     y="Delivery Gap",
119     color="Shipping Mode",
120     title="Average Delivery Gap by Shipping Mode"
121 )
122 fig.update_layout(xaxis_tickangle=-45)
123 st.plotly_chart(fig, width="stretch")
124
125 region_gap = (
126     filtered_df.groupby("Order Region")["Delivery Gap"]
127     .mean()
128     .reset_index()
129     .sort_values("Delivery Gap", ascending=False)
130 )
131
132 fig = px.bar(
133     region_gap,
134     x="Order Region",
135     y="Delivery Gap",
136     color="Delivery Gap",
137     title="Average Delivery Gap by Region"
138 )
```

```
140
141
142 st.plotly_chart(fig, width="stretch")
143
144 st.markdown("----")
145 st.subheader("📄 Dataset Preview")
146
147 st.dataframe(filtered_df.head(100))
148
149
150 st.download_button(
151     "📄 Download CSV",
152     filtered_df.to_csv(index=False),
153     file_name="Filtered_Data.csv",
154     mime="text/csv"
155 )
156
157
```

```
app.py > ...
98 fig = px.pie(
99     status,
100     names="Delivery Status",
101     values="Count",
102     title="Delivery Status Distribution",
103     hole=0.4
104 )
105
```

```
app.py > ...
35
36 # Sidebar
37 # -----
38
39 st.sidebar.header("Filters")
40
41 shipping = st.sidebar.multiselect(
42     "Shipping Mode",
43     df["Shipping Mode"].unique(),
44     default=df["Shipping Mode"].unique()
45 )
46
47 region = st.sidebar.multiselect(
48     "Region",
49     df["Order Region"].unique(),
50     default=df["Order Region"].unique()
51 )
52
53 filtered_df = df[
54     (df["Shipping Mode"].isin(shipping)) &
55     (df["Order Region"].isin(region))
56 ]
57
58 # -----
59 # API Calls
60 # -----
61 total_orders = len(filtered_df)
62
63 avg_gap = filtered_df["Delivery Gap"].mean()
64
65 late_orders = len(filtered_df[filtered_df["Delivery Gap"] > 0])
66
67 late_ratio = (late_orders / total_orders) * 100
68
```



GitHub Repository :-

iamkaran1735 / Supply-Chain-Analytics-Dashboard

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Supply-Chain-Analytics-Dashboard Public

main 1 Branch 0 Tags

Go to file

Add file Code

iamkaran1735 Added requirements.txt ✓ ec7b105 · 26 minutes ago 2 Commits

- data Initial commit yesterday
- images Initial commit yesterday
- notebooks Initial commit yesterday
- app.py Initial commit yesterday
- requirements.txt Added requirements.txt 26 minutes ago

README

Add a README

About

No description, website, or topics provided.

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

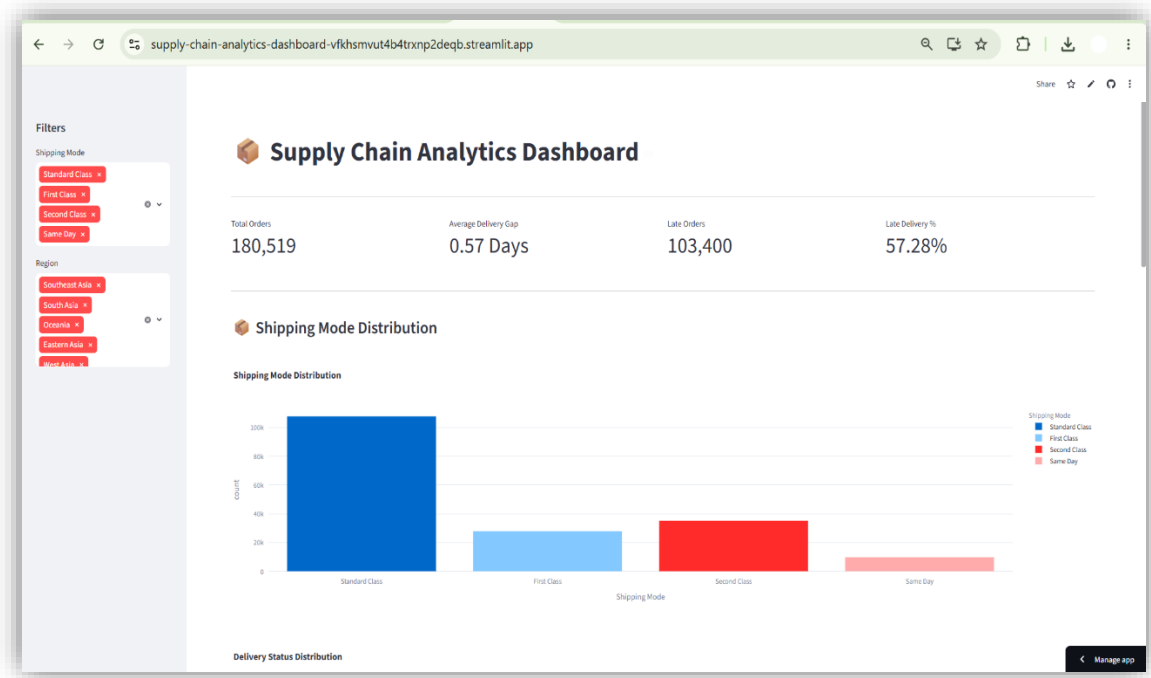
Deployments 2

github-pages 25 minutes ago

Packages



Streamlit Deployment :-



The Streamlit deployment confirms that the Supply Chain Analytics Dashboard is successfully hosted as a web application. Users can access the dashboard through a web browser and utilize features such as interactive filters, KPI cards, visualizations, dataset preview, and CSV download without installing any additional software.

Conclusion :-

The **Supply Chain Analytics Dashboard** developed in this project successfully demonstrates how data analytics and visualization can improve supply chain management and support better decision-making. The dashboard provides an interactive platform for analyzing logistics data using **Python, Streamlit, Pandas, and Plotly**. It enables users to monitor important Key Performance Indicators (KPIs), analyze shipping modes, evaluate delivery status, compare regional performance, and calculate the average delivery gap.

The implementation of interactive charts, filters, and downloadable reports makes the dashboard user-friendly and efficient for exploring large datasets. By transforming raw supply chain data into meaningful visual insights, the dashboard helps organizations identify delivery delays, optimize shipping performance, and improve operational efficiency.

Overall, this project demonstrates the practical application of data analytics in supply chain management. It provides a simple, scalable, and effective solution for analyzing logistics performance and supports data-driven decision-making for improving business operations.

Future Scope :-

- Machine Learning
- AI Prediction
- Live API
- Inventory Forecasting
- Route Optimization

References :-

1. Streamlit Documentation
2. Plotly Documentation
3. Pandas Documentation
4. Python Documentation
5. DataCo Supply Chain Dataset